


```

!pip install --quiet jax jaxlib # Only needed if JAX not present

import jax
import jax.numpy as jnp
import numpy as np
from jax.scipy.linalg import expm

# =====
# SU(3) basis and exp map
# =====

def su3_generators():
    """
    ... Return 8 anti-Hermitian su(3) generators T_a (3x3 complex),
    ... built from the standard Gell-Mann matrices:
    ... .. T_a = i * lambda_a / 2
    ... """
    ... lam = []

    ... lam1 = jnp.array([[0,1,0],[1,0,0],[0,0,0]], dtype=jnp.complex64)
    ... lam2 = jnp.array([[0,-1j,0],[1j,0,0],[0,0,0]], dtype=jnp.complex64)
    ... lam3 = jnp.array([[1,0,0],[0,-1,0],[0,0,0]], dtype=jnp.complex64)
    ... lam4 = jnp.array([[0,0,1],[0,0,0],[1,0,0]], dtype=jnp.complex64)
    ... lam5 = jnp.array([[0,0,-1j],[0,0,0],[1j,0,0]], dtype=jnp.complex64)
    ... lam6 = jnp.array([[0,0,0],[0,0,1],[0,1,0]], dtype=jnp.complex64)
    ... lam7 = jnp.array([[0,0,0],[0,0,-1j],[0,1j,0]], dtype=jnp.complex64)
    ... lam8 = jnp.array([[1,0,0],[0,1,0],[0,0,-2]], dtype=jnp.complex64) / jnp.sc

    ... lam = jnp.stack([lam1, lam2, lam3, lam4, lam5, lam6, lam7, lam8], axis=0)

    ... # Anti-Hermitian generators T_a = i * lam_a / 2
    ... T = 1j * lam / 2.0
    ... return T # shape (8,3,3)

T_SU3 = su3_generators() # global

def su3_alg_from_vec(a_vec):
    """
    ... Given a_vec in R^8, build A = sum_a a_vec[a] * T_a (3x3 anti-Hermitian).
    ... a_vec may have shape (... ,8).
    ... """
    ... return jnp.einsum("...a,aij->...ij", a_vec, T_SU3)

def su3_exp(a_vec):
    """
    ... Compute U = exp(A) for A in su(3) constructed from 8 real params.
    ... a_vec shape (8,) or (... ,8).
    ... """
    ... A = su3_alg_from_vec(a_vec) # (... ,3,3)

```

```

    ... return expm(A)

# =====
# Lattice indexing helpers
# =====

def shift_idx(idx, mu, L):
    """
    ... Periodic shift index idx = (x,y,z,t) in direction mu.
    ... mu in {0,1,2,3}.
    ... """
    ... x, y, z, t = idx
    ... if mu == 0:
    ...     return ((x+1) % L, y, z, t)
    ... elif mu == 1:
    ...     return (x, (y+1) % L, z, t)
    ... elif mu == 2:
    ...     return (x, y, (z+1) % L, t)
    ... elif mu == 3:
    ...     return (x, y, z, (t+1) % L)
    ... else:
    ...     raise ValueError("mu must be 0,1,2,3")

def all_sites(L):
    ... return [(x,y,z,t)
    ...         for x in range(L)
    ...         for y in range(L)
    ...         for z in range(L)
    ...         for t in range(L)]

# =====
# Build link matrices from params
# =====

def build_links_from_params(params, L):
    """
    ... params: real array of shape (L,L,L,L,4,8)
    ...     Each link has 8 real parameters (su(3) coordinates).
    ... Returns:
    ...     U: complex array (L,L,L,L,4,3,3)
    ... """
    ... flat = params.reshape(-1, 8) # (N_links, 8)
    ... U_flat = jax.vmap(su3_exp)(flat) # (N_links, 3,3)
    ... U = U_flat.reshape(L, L, L, L, 4, 3, 3)
    ... return U

# =====
# Wilson action
# =====

def wilson_action(params, L, beta):

```

```

... .._.....(params, L, .....
... """
... Wilson action for SU(3) on L^4 lattice.
... params: (L,L,L,L,4,8) real
... """

... U = build_links_from_params(params, L) # (L,L,L,L,4,3,3)
... action = 0.0
... for x, y, z, t in all_sites(L):
...     for mu in range(4):
...         for nu in range(mu+1, 4):
...             idx = (x, y, z, t)
...             x_mu = shift_idx(idx, mu, L)
...             x_nu = shift_idx(idx, nu, L)

...             # links:
...             U1 = U[x, y, z, t, mu]
...             U2 = U[x_mu[0], x_mu[1], x_mu[2], x_mu[3], nu]
...             U3 = jnp.conjugate(U[x_nu[0], x_nu[1], x_nu[2], x_nu[3], mu]).
...             U4 = jnp.conjugate(U[x, y, z, t, nu].T)

...             Uplaq = U1 @ U2 @ U3 @ U4
...             action += beta * (1.0 - (1.0/3.0) * jnp.real(jnp.trace(Uplaq)))
... return action

# =====
# Haar mass term (quadratic)
# =====

def haar_mass_term_exact(params, c0):
    """
    Haar mass term using the quadratic expansion:
    .....  $S_{\text{Haar}}^{\{(2)\}}(A) \sim c0 * \text{Tr}(A^2)$ ,
    ..... implemented as  $c0 * \text{Tr}(A^\dagger A)$  per link (positive definite).
    ..... params: (L,L,L,L,4,8)
    ..... """
    ..... flat = params.reshape(-1, 8) # (N_links, 8)

    ..... def per_link(a_vec):
    .....     A = su3_alg_from_vec(a_vec) ..... # (3,3) anti-Hermitian
    .....     tr_AA = jnp.real(jnp.trace(A.conj().T @ A))
    .....     return tr_AA

    ..... tr_vals = jax.vmap(per_link)(flat) ..... # (N_links,)
    ..... return c0 * jnp.sum(tr_vals)

def total_action(params, L, beta, c0):
    """
    ..... Combined action: Wilson + Haar quadratic mass term.
    ..... params: (L,L,L,L,4,8)
    ..... """
    ..... S_W = wilson_action(params, L, beta)

```

```

... S_H = haar_mass_term_exact(params, c0)
... return S_W + S_H

# =====
# Flattened interface for Hessian
# =====

def make_flat_funcs(L, beta, c0):
    """
    ... Return:
    ... flat_action:  $R^{(n\_params)}$  -> scalar
    ... unflatten:  $R^{(n\_params)}$  -> (L,L,L,L,4,8)
    ... n_params: int
    ... """
    ... sample = jnp.zeros((L, L, L, L, 4, 8), dtype=jnp.float32)
    ... n_params = sample.size

    ... def unflatten(theta_flat):
    ...     return theta_flat.reshape((L, L, L, L, 4, 8))

    ... def flat_action(theta_flat):
    ...     params = unflatten(theta_flat)
    ...     return total_action(params, L=L, beta=beta, c0=c0)

    ... return flat_action, unflatten, n_params

# =====
# Hessian computation
# =====

def compute_hessian(L=2, beta=2.0, c0=0.25):
    """
    ... Compute the full Hessian of the total action at  $\theta_0 = 0$ .
    ... WARNING: Hessian size =  $(L^4 * 4 * 8)^2$ .
    ... For L=2:  $16*4*8=512$  params -> 512x512 Hessian (OK).
    ... For larger L this blows up fast.
    ... """
    ... flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)

    ... print(f"L={L}, n_params={n_params}")

    ... theta0 = jnp.zeros((n_params,), dtype=jnp.float32)

    ... # Build Hessian via jax.hessian
    ... hess_fun = jax.hessian(flat_action)
    ... H = hess_fun(theta0) # shape (n_params, n_params)

    ... return H

# =====
# Example run

```

```
# =====

L = 2 # lattice linear size
beta = 2.0 # Wilson coupling
c0 = 0.25 # Haar quadratic coefficient ~ N/12 for SU(3), check your norm

print("Computing Hessian for SU(3) Wilson+Haar on L^4 lattice...")
H = compute_hessian(L=L, beta=beta, c0=c0)

print("Hessian shape:", H.shape)

# Convert to numpy and compute eigenvalues (small example only!)
H_np = np.array(H, dtype=np.float64)
evals = np.linalg.eigvalsh(H_np)
print("Smallest 10 eigenvalues of Hessian:")
print(evals[:10])
```

```
<>:132: SyntaxWarning: invalid escape sequence '\d'
<>:132: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipython-input-1016489893.py:132: SyntaxWarning: invalid escape sequence
implemented as c0 * Tr(A^\dagger A) per link (positive definite).
Computing Hessian for SU(3) Wilson+Haar on L^4 lattice...
L=2, n_params=512
/usr/local/lib/python3.12/dist-packages/jax/_src/lax/lax.py:5473: ComplexWarn
x_bar = _convert_element_type(x_bar, x.aval.dtype, x.aval.weak_type)
Hessian shape: (512, 512)
Smallest 10 eigenvalues of Hessian:
[0.24999976 0.24999976 0.24999976 0.24999976 0.24999976 0.24999976
 0.24999976 0.24999976 0.24999976 0.24999976]
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
!pip install --quiet jax jaxlib
```

```
import jax
import jax.numpy as jnp
import numpy as np
from jax.scipy.linalg import expm
```

```
# =====
# SU(3) basis and exp map
# =====
```

```
def su3_generators():
```

```
    """
```

```
    Return 8 anti-Hermitian su(3) generators T_a (3x3 complex),
    built from the standard Gell-Mann matrices:
```

```
    T_a = i * lambda_a / 2
```

```

    T_a = 1j * lam_a / 2
    """
    lam = []

    lam1 = jnp.array([[0,1,0],[1,0,0],[0,0,0]], dtype=jnp.complex64)
    lam2 = jnp.array([[0,-1j,0],[1j,0,0],[0,0,0]], dtype=jnp.complex64)
    lam3 = jnp.array([[1,0,0],[0,-1,0],[0,0,0]], dtype=jnp.complex64)
    lam4 = jnp.array([[0,0,1],[0,0,0],[1,0,0]], dtype=jnp.complex64)
    lam5 = jnp.array([[0,0,-1j],[0,0,0],[1j,0,0]], dtype=jnp.complex64)
    lam6 = jnp.array([[0,0,0],[0,0,1],[0,1,0]], dtype=jnp.complex64)
    lam7 = jnp.array([[0,0,0],[0,0,-1j],[0,1j,0]], dtype=jnp.complex64)
    lam8 = jnp.array([[1,0,0],[0,1,0],[0,0,-2]], dtype=jnp.complex64) / jnp.sc

    lam = jnp.stack([lam1, lam2, lam3, lam4, lam5, lam6, lam7, lam8], axis=0)

    # Anti-Hermitian generators T_a = i * lam_a / 2
    T = 1j * lam / 2.0
    return T # shape (8,3,3)

T_SU3 = su3_generators()

def su3_alg_from_vec(a_vec):
    """
    Given a_vec in R^8, build A = sum_a a_vec[a] * T_a (3x3 anti-Hermitian).
    a_vec may have shape (... ,8).
    """
    return jnp.einsum("...a,aij->...ij", a_vec, T_SU3)

def su3_exp(a_vec):
    """
    Compute U = exp(A) for A in su(3) constructed from 8 real params.
    a_vec shape (8,) or (... ,8).
    """
    A = su3_alg_from_vec(a_vec) # (... ,3,3)
    return expm(A)

# =====
# Lattice indexing helpers
# =====

def shift_idx(idx, mu, L):
    """
    Periodic shift index idx = (x,y,z,t) in direction mu.
    mu in {0,1,2,3}.
    """
    x, y, z, t = idx
    if mu == 0:
        return ((x + 1) % L, y, z, t)
    elif mu == 1:
        return (x, (y + 1) % L, z, t)
    elif mu == 2:

```

```

        return (x, y, (z + 1) % L, t)
    elif mu == 3:
        return (x, y, z, (t + 1) % L)
    else:
        raise ValueError("mu must be 0,1,2,3")

def all_sites(L):
    return [(x,y,z,t)
            for x in range(L)
            for y in range(L)
            for z in range(L)
            for t in range(L)]

# =====
# Build link matrices from params
# =====

def build_links_from_params(params, L):
    """
    params: real array of shape (L,L,L,L,4,8)
            Each link has 8 real parameters (su(3) coordinates).
    Returns:
        U: complex array (L,L,L,L,4,3,3)
    """
    flat = params.reshape(-1, 8)                # (N_links, 8)
    U_flat = jax.vmap(su3_exp)(flat)            # (N_links, 3,3)
    U = U_flat.reshape(L, L, L, L, 4, 3, 3)
    return U

# =====
# Wilson action
# =====

def wilson_action(params, L, beta):
    """
    Wilson action for SU(3) on L^4 lattice.
    params: (L,L,L,L,4,8) real
    """
    U = build_links_from_params(params, L) # (L,L,L,L,4,3,3)
    action = 0.0
    for x, y, z, t in all_sites(L):
        for mu in range(4):
            for nu in range(mu+1, 4):
                idx = (x, y, z, t)
                x_mu = shift_idx(idx, mu, L)
                x_nu = shift_idx(idx, nu, L)

                # links:
                U1 = U[x, y, z, t, mu]
                U2 = U[x_mu[0], x_mu[1], x_mu[2], x_mu[3], nu]
                U3 = jnp.conj(U1 * U2)

```



```

        U3 = jnp.conjugate(U[x_nu[0], x_nu[1], x_nu[2], x_nu[3], nu])
        U4 = jnp.conjugate(U[x, y, z, t, nu].T)

        Uplaq = U1 @ U2 @ U3 @ U4
        action += beta * (1.0 - (1.0/3.0) * jnp.real(jnp.trace(Uplaq)))
    return action

# =====
# Haar mass term (quadratic)
# =====

def haar_mass_term_exact(params, c0):
    """
    Haar mass term using the quadratic expansion:
     $S_{\text{Haar}}^{\{(2)\}}(A) \sim c0 * \text{Tr}(A^2)$ ,
    implemented as  $c0 * \text{Tr}(A^\dagger A)$  per link (positive definite).

    params: (L,L,L,L,4,8)
    """
    flat = params.reshape(-1, 8) # (N_links, 8)

    def per_link(a_vec):
        A = su3_alg_from_vec(a_vec) # (3,3) anti-Hermitian
        tr_AA = jnp.real(jnp.trace(A.conj().T @ A))
        return tr_AA

    tr_vals = jax.vmap(per_link)(flat) # (N_links,)
    return c0 * jnp.sum(tr_vals)

def total_action(params, L, beta, c0):
    """
    Combined action: Wilson + Haar quadratic mass term.
    params: (L,L,L,L,4,8)
    """
    S_W = wilson_action(params, L, beta)
    S_H = haar_mass_term_exact(params, c0)
    return S_W + S_H

# =====
# Flattened interface for Hessian
# =====

def make_flat_funcs(L, beta, c0):
    """
    Return:
    flat_action:  $R^{(n_{\text{params}})} \rightarrow \text{scalar}$ 
    unflatten:  $R^{(n_{\text{params}})} \rightarrow (L,L,L,L,4,8)$ 
    n_params: int
    """
    sample = jnp.zeros((L, L, L, L, 4, 8), dtype=jnp.float32)
    n_params = sample.size

```

```

def unflatten(theta_flat):
    return theta_flat.reshape((L, L, L, L, 4, 8))

def flat_action(theta_flat):
    params = unflatten(theta_flat)
    return total_action(params, L=L, beta=beta, c0=c0)

return flat_action, unflatten, n_params

# =====
# Hessian computation
# =====

def compute_hessian(L=2, beta=2.0, c0=0.25):
    """
    Compute the full Hessian of the total action at theta0 = 0.
    WARNING: Hessian size = (L^4 * 4 * 8)^2.
    For L=2: 16*4*8=512 params -> 512x512 Hessian.
    """
    flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)

    print(f"L={L}, n_params={n_params}")

    theta0 = jnp.zeros((n_params,), dtype=jnp.float32)

    # JIT-compile the action for speed
    flat_action_jit = jax.jit(flat_action)

    # Build Hessian via jax.hessian of the jitted action
    hess_fun = jax.hessian(flat_action_jit)
    H = hess_fun(theta0)  # shape (n_params, n_params)

    return H

# =====
# Example run
# =====

L = 2          # lattice linear size
beta = 2.0     # Wilson coupling
c0 = 0.25     # Haar quadratic coefficient, check normalization later

print("Computing Hessian for SU(3) Wilson+Haar on L^4 lattice...")
H = compute_hessian(L=L, beta=beta, c0=c0)

print("Hessian shape:", H.shape)

# Convert to numpy and compute eigenvalues (for small problems only!)
H_np = np.array(H, dtype=np.float64)
evals = np.linalg.eigvalsh(H_np)

```

```
evals = np.linalg.eigvalsh(H_np)
evals_sorted = np.sort(evals)
```

```
print("Smallest 10 eigenvalues of Hessian:")
print(evals_sorted[:10])
```

Computing Hessian for SU(3) Wilson+Haar on L^4 lattice...

L=2, n_params=512

Hessian shape: (512, 512)

Smallest 10 eigenvalues of Hessian:

```
[0.24999976 0.24999976 0.24999976 0.24999976 0.24999976 0.24999976
 0.24999976 0.24999976 0.24999976 0.24999976]
```

```
import time
```

```
def min_eig_at(theta_flat, L, beta, c0):
```

```
    """Compute smallest eigenvalue of Hessian at given theta_flat."""
```

```
    flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)
```

```
    flat_action_jit = jax.jit(flat_action)
```

```
    hess_fun = jax.hessian(flat_action_jit)
```

```
    H = hess_fun(theta_flat)
```

```
    H_np = np.array(H, dtype=np.float64)
```

```
    evals = np.linalg.eigvalsh(H_np)
```

```
    return evals.min()
```

```
def sample_min_eigs(
```

```
    L=2, beta=2.0, c0=0.25,
```

```
    n_samples=5, sigma=0.1, key_seed=0
```

```
):
```

```
    """
```

```
    Sample random configs  $\theta_0 \sim N(0, \sigma^2)$ ,
```

```
    compute smallest eigenvalue of Hessian at each.
```

```
    """
```

```
    flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)
```

```
    print(f"Sampling {n_samples} configs in  $R^{n\_params}$ ...")
```

```
    key = jax.random.PRNGKey(key_seed)
```

```
    mins = []
```

```
    for i in range(n_samples):
```

```
        key, subkey = jax.random.split(key)
```

```
        theta0 = sigma * jax.random.normal(subkey, (n_params,), dtype=jnp.fl
```

```
        t0 = time.time()
```

```
        lam_min = min_eig_at(theta0, L, beta, c0)
```

```
        t1 = time.time()
```

```
        mins.append(lam_min)
```

```
        print(f"Sample {i}: lambda_min = {lam_min:.6f} (time {t1-t0:.2f}s)"
```

```
    return np.array(mins)
```

```
# Example: try a few random configs near zero
```

```
mins = sample_min_eigs(L=2, beta=2.0, c0=0.25, n_samples=3, sigma=0.05, key_
```

```
print("Min eigenvalues over samples:", mins)
print("Global min:", mins.min())
```

```
Sampling 3 configs in R^512...
Sample 0: lambda_min = 0.124628 (time 53.03s)
Sample 1: lambda_min = 0.101073 (time 54.45s)
Sample 2: lambda_min = 0.136030 (time 53.76s)
Min eigenvalues over samples: [0.12462767 0.10107274 0.1360303 ]
Global min: 0.10107274161283288
```

```
import time

def min_eig_at(theta_flat, L, beta, c0):
    """
    Compute smallest eigenvalue of Hessian at given theta_flat.
    WARNING: Very expensive for large n_params.
    """
    flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)
    flat_action_jit = jax.jit(flat_action)
    hess_fun = jax.hessian(flat_action_jit)
    H = hess_fun(theta_flat)
    H_np = np.array(H, dtype=np.float64)
    evals = np.linalg.eigvalsh(H_np)
    return evals.min()

def sample_min_eigs_for_sigma(
    L, beta, c0,
    sigma, n_samples=3, key_seed=0
):
    """
    For a fixed sigma, sample  $\theta_0 \sim N(0, \sigma^2)$  in  $R^{\{n\_params\}}$ ,
    compute smallest eigenvalue of Hessian at each sample.
    """
    flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)
    print(f"\n[σ={sigma}] Sampling {n_samples} configs in R^{{n_params}}...")

    key = jax.random.PRNGKey(key_seed)
    mins = []
    for i in range(n_samples):
        key, subkey = jax.random.split(key)
        theta0 = sigma * jax.random.normal(subkey, (n_params,), dtype=jnp.fl
        t0 = time.time()
        lam_min = min_eig_at(theta0, L, beta, c0)
        t1 = time.time()
        mins.append(lam_min)
        print(f" Sample {i}: lambda_min = {lam_min:.6f} (time {t1-t0:.2f}s
    mins = np.array(mins)
    print(f" → σ={sigma}: min={mins.min():.6f}, mean={mins.mean():.6f}")
    return mins

def sweep sigma(
```

```

def sweep_sigma(
    L=2, beta=2.0, c0=0.25,
    sigma_list=(0.0, 0.02, 0.05, 0.1, 0.2),
    n_samples=3, key_seed_base=0
):
    """
    Sweep over sigma values, collecting smallest eigenvalues at each.
    Returns:
        results: dict sigma -> array of lambda_mins
    """
    results = {}
    for k, sigma in enumerate(sigma_list):
        mins = sample_min_eigs_for_sigma(
            L=L, beta=beta, c0=c0,
            sigma=sigma, n_samples=n_samples,
            key_seed=key_seed_base + k
        )
        results[sigma] = mins
    return results

# =====
# Run  $\sigma$  sweep at fixed  $\beta$ 
# =====

L = 2
beta = 2.0
c0 = 0.25

sigma_list = [0.0, 0.02, 0.05, 0.1, 0.2] # you can extend this
n_samples = 3

results = sweep_sigma(
    L=L, beta=beta, c0=c0,
    sigma_list=sigma_list,
    n_samples=n_samples,
    key_seed_base=123
)

print("\nSummary:")
for sigma in sigma_list:
    mins = results[sigma]
    print(f" $\sigma$ ={sigma:5.3f}  min  $\lambda_{\min}$ ={mins.min():.6f}  mean  $\lambda_{\min}$ ={mins.me
```

```

[ $\sigma$ =0.0] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.250000 (time 55.69s)
Sample 1: lambda_min = 0.250000 (time 59.04s)
Sample 2: lambda_min = 0.250000 (time 57.60s)
→  $\sigma$ =0.0: min=0.250000, mean=0.250000
```

```

[ $\sigma$ =0.02] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.206524 (time 57.47s)
```

```

Sample 1: lambda_min = 0.195158 (time 56.45s)
Sample 2: lambda_min = 0.199812 (time 56.67s)
→  $\sigma=0.02$ : min=0.195158, mean=0.200498

```

```

[ $\sigma=0.05$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.129786 (time 57.70s)
Sample 1: lambda_min = 0.120242 (time 57.57s)
Sample 2: lambda_min = 0.120261 (time 56.10s)
→  $\sigma=0.05$ : min=0.120242, mean=0.123430

```

```

[ $\sigma=0.1$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = -0.021118 (time 57.16s)
Sample 1: lambda_min = -0.017744 (time 56.41s)
Sample 2: lambda_min = -0.026227 (time 57.36s)
→  $\sigma=0.1$ : min=-0.026227, mean=-0.021696

```

```

[ $\sigma=0.2$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = -0.383596 (time 55.90s)
Sample 1: lambda_min = -0.291215 (time 59.22s)
Sample 2: lambda_min = -0.361844 (time 59.13s)
→  $\sigma=0.2$ : min=-0.383596, mean=-0.345552

```

Summary:

```

 $\sigma=0.000$  min  $\lambda_{\min}=0.250000$  mean  $\lambda_{\min}=0.250000$ 
 $\sigma=0.020$  min  $\lambda_{\min}=0.195158$  mean  $\lambda_{\min}=0.200498$ 
 $\sigma=0.050$  min  $\lambda_{\min}=0.120242$  mean  $\lambda_{\min}=0.123430$ 
 $\sigma=0.100$  min  $\lambda_{\min}=-0.026227$  mean  $\lambda_{\min}=-0.021696$ 
 $\sigma=0.200$  min  $\lambda_{\min}=-0.383596$  mean  $\lambda_{\min}=-0.345552$ 

```

```

import time

def sweep_beta_sigma(
    L=2,
    beta_list=(0.5, 1.0, 2.0, 4.0),
    c0=0.25,
    sigma_list=(0.0, 0.02, 0.05, 0.1, 0.2),
    n_samples=3,
    key_seed_base=1000
):
    """
    Sweep over beta values and sigma amplitudes.
    For each (beta, sigma):
        - sample n_samples random configs  $\theta \sim N(0, \sigma^2)$ ,
        - compute the smallest eigenvalue of the Hessian.
    Returns:
        results[beta][sigma] = array of lambda_min's.
    """
    results = {}
    for j, beta in enumerate(beta_list):
        print("\n" + "="*60)
        print(f"Starting  $\beta={\text{beta}}$ ")
        print("="*60)
        beta_results = []

```

```

beta_results = {}
for k, sigma in enumerate(sigma_list):
    # Reuse sample_min_eigs_for_sigma, but with per-(beta,sigma) see
    mins = sample_min_eigs_for_sigma(
        L=L, beta=beta, c0=c0,
        sigma=sigma, n_samples=n_samples,
        key_seed=key_seed_base + 10*j + k
    )
    beta_results[sigma] = mins
results[beta] = beta_results

return results

# =====
# Run  $\beta$ - $\sigma$  sweep
# =====

L = 2
c0 = 0.25

beta_list = [0.5, 1.0, 2.0, 4.0]          # you can change this set
sigma_list = [0.0, 0.02, 0.05, 0.1, 0.2] # same  $\sigma$  grid as before
n_samples = 3

results_beta_sigma = sweep_beta_sigma(
    L=L,
    beta_list=beta_list,
    c0=c0,
    sigma_list=sigma_list,
    n_samples=n_samples,
    key_seed_base=2025
)

print("\n\nGLOBAL SUMMARY:")
for beta in beta_list:
    print(f"\n $\beta$  = {beta}")
    for sigma in sigma_list:
        mins = results_beta_sigma[beta][sigma]
        print(f"   $\sigma$ = $\{sigma:5.3f\}$   min  $\lambda_{\min}=\{mins.min():.6f\}$   mean  $\lambda_{\min}=\{m$ 

```

```

=====
Starting  $\beta=0.5$ 
=====

```

```

[ $\sigma=0.0$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0:  $\lambda_{\min}$  = 0.250000 (time 60.26s)
Sample 1:  $\lambda_{\min}$  = 0.250000 (time 56.35s)
Sample 2:  $\lambda_{\min}$  = 0.250000 (time 57.67s)
→  $\sigma=0.0$ : min=0.250000, mean=0.250000

```

```

[ $\sigma=0.02$ ] Sampling 3 configs in  $R^{512}$ ...

```

```

Sample 0: lambda_min = 0.237264 (time 56.26s)
Sample 1: lambda_min = 0.238302 (time 56.82s)
Sample 2: lambda_min = 0.237995 (time 58.12s)
→  $\sigma=0.02$ : min=0.237264, mean=0.237854

```

```

[ $\sigma=0.05$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.216828 (time 56.90s)
Sample 1: lambda_min = 0.218943 (time 58.81s)
Sample 2: lambda_min = 0.216981 (time 57.33s)
→  $\sigma=0.05$ : min=0.216828, mean=0.217584

```

```

[ $\sigma=0.1$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.182550 (time 59.44s)
Sample 1: lambda_min = 0.189814 (time 55.42s)
Sample 2: lambda_min = 0.186126 (time 59.91s)
→  $\sigma=0.1$ : min=0.182550, mean=0.186163

```

```

[ $\sigma=0.2$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.097039 (time 56.69s)
Sample 1: lambda_min = 0.108310 (time 60.22s)
Sample 2: lambda_min = 0.110610 (time 56.98s)
→  $\sigma=0.2$ : min=0.097039, mean=0.105320

```

```

=====
Starting  $\beta=1.0$ 
=====

```

```

[ $\sigma=0.0$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.250000 (time 57.08s)
Sample 1: lambda_min = 0.250000 (time 60.57s)
Sample 2: lambda_min = 0.250000 (time 56.58s)
→  $\sigma=0.0$ : min=0.250000, mean=0.250000

```

```

[ $\sigma=0.02$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.226647 (time 60.07s)
Sample 1: lambda_min = 0.224447 (time 57.46s)
Sample 2: lambda_min = 0.224392 (time 56.47s)
→  $\sigma=0.02$ : min=0.224392, mean=0.225162

```

```

[ $\sigma=0.05$ ] Sampling 3 configs in  $R^{512}$ ...
Sample 0: lambda_min = 0.185950 (time 60.49s)
Sample 1: lambda_min = 0.185171 (time 56.70s)
Sample 2: lambda_min = 0.187222 (time 56.35s)
→  $\sigma=0.05$ : min=0.185171, mean=0.186114

```

```

[ $\sigma=0.1$ ] Sampling 3 configs in  $R^{512}$ ...

```

```

def metropolis_sample(
    L=2,
    beta=2.0,
    c0=0.25,
    step_size=0.01,
    n_steps=200,
    burn_in=50,
    seed=123

```



```

seed=123,
verbose=True,
):
    """
    Simple Metropolis sampler in A-coordinates:
         $\theta \rightarrow \theta + \eta, \quad \eta \sim N(0, \text{step\_size}^2)$ 
    Accept with prob  $\min(1, \exp(-\Delta S))$ .
    After burn_in, record  $||\theta||$ .
    """
    flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)

    key = jax.random.PRNGKey(seed)
    theta = jnp.zeros((n_params,), dtype=jnp.float32)
    S = action_at(theta, L, beta, c0)

    samples_norm = []
    samples_theta = []

    for i in range(n_steps):
        key, subkey = jax.random.split(key)
        proposal = theta + step_size * jax.random.normal(subkey, (n_params,))
        S_prop = action_at(proposal, L, beta, c0)
        dS = S_prop - S

        accept = (dS < 0) or (np.random.rand() < np.exp(-dS))
        if accept:
            theta = proposal
            S = S_prop

        if i >= burn_in:
            nrm = field_norm(theta)
            samples_norm.append(nrm)
            samples_theta.append(np.array(theta))
            if verbose:
                print(f"Step {i:4d}, norm={nrm:.5f}, S={S:.5f}")
        else:
            if verbose:
                # No sample recorded yet, just report S
                print(f"Step {i:4d}, (burn-in), S={S:.5f}")

    return np.array(samples_norm), samples_theta

```

```

L = 2
beta = 2.0
c0 = 0.25

samples_norm, samples_theta = metropolis_sample(
    L=L,
    beta=beta,
    c0=c0,
    . . .

```

```

    step_size=0.01,
    n_steps=50,      # bump to 200+ for more data
    burn_in=10,
    seed=2025,
    verbose=True
)

print("\nCollected norms:")
print(samples_norm)

print("\nStatistics:")
print(" mean |A| =", samples_norm.mean())
print(" max |A|  =", samples_norm.max())
print(" min |A|  =", samples_norm.min())

```

```

Step    0, (burn-in), S=0.04868
Step    1, (burn-in), S=0.11710
Step    2, (burn-in), S=0.11710
Step    3, (burn-in), S=0.16203
Step    4, (burn-in), S=0.21643
Step    5, (burn-in), S=0.27441
Step    6, (burn-in), S=0.32277
Step    7, (burn-in), S=0.36841
Step    8, (burn-in), S=0.42006
Step    9, (burn-in), S=0.49051
Step   10, norm=0.64618, S=0.49051
Step   11, norm=0.69390, S=0.54834
Step   12, norm=0.73112, S=0.60990
Step   13, norm=0.77691, S=0.68631
Step   14, norm=0.80851, S=0.74603
Step   15, norm=0.84484, S=0.80887
Step   16, norm=0.88101, S=0.85684
Step   17, norm=0.90410, S=0.89419
Step   18, norm=0.93877, S=0.96974
Step   19, norm=0.97244, S=1.04749
Step   20, norm=1.00159, S=1.11856
Step   21, norm=1.01670, S=1.15195
Step   22, norm=1.03634, S=1.19133
Step   23, norm=1.05626, S=1.26051
Step   24, norm=1.07288, S=1.31423
Step   25, norm=1.09981, S=1.37571
Step   26, norm=1.11477, S=1.42474
Step   27, norm=1.15708, S=1.52425
Step   28, norm=1.18331, S=1.55923
Step   29, norm=1.21799, S=1.62473
Step   30, norm=1.24557, S=1.72169
Step   31, norm=1.26124, S=1.73689
Step   32, norm=1.28853, S=1.80904
Step   33, norm=1.29191, S=1.81437
Step   34, norm=1.29926, S=1.85331
Step   35, norm=1.30060, S=1.87761
Step   36, norm=1.30658, S=1.88018
Step   37, norm=1.33066, S=1.95179
Step   38, norm=1.34744, S=2.03160
..    ..

```

```

Step 39, norm=1.38296, S=2.09433
Step 40, norm=1.38529, S=2.12228
Step 41, norm=1.40124, S=2.17018
Step 42, norm=1.41715, S=2.23915
Step 43, norm=1.44423, S=2.33472
Step 44, norm=1.45264, S=2.36448
Step 45, norm=1.48779, S=2.48135
Step 46, norm=1.51170, S=2.54337
Step 47, norm=1.52455, S=2.54404
Step 48, norm=1.53245, S=2.56785
Step 49, norm=1.54377, S=2.58866

```

Collected norms:

```

[0.64618176 0.69390154 0.73111594 0.77690893 0.80851406 0.84483743
 0.88101196 0.90409982 0.93876702 0.97243607 1.00159383 1.01669848
 1.03634107 1.05626452 1.07287586 1.09981024 1.11476684 1.15708363
 1.18331277 1.21799254 1.24556971 1.26123989 1.28852737 1.29190719
 1.29926383 1.30059719 1.30657971 1.33065724 1.34744024 1.38295972
 1.3852948 1.40124297 1.41715181 1.44423497 1.4526403 1.48779452

```

```

import time

# =====
# Parameters for the sweep
# =====

L = 2
c0 = 0.25

beta_list = [0.5, 1.0, 2.0, 4.0]
sigma_grid = [0.0, 0.01, 0.02, 0.03, 0.05, 0.07, 0.10, 0.15, 0.20]
n_samples = 2 # 2 is cheap; use 3 if you want more stability

results = {}

print("Starting  $\beta$ - $\sigma$  Hessian convexity sweep...\n")

for beta in beta_list:
    print("="*70)
    print(f" $\beta$  = {beta}")
    print("="*70)
    beta_results = {}
    for j, sigma in enumerate(sigma_grid):
        t0 = time.time()
        lam_min, lam_mean = estimate_lambda_min(
            L=L,
            beta=beta,
            c0=c0,
            sigma=sigma,
            n_samples=n_samples,
            seed=1000 + int(100*beta) + j
        )
        .. .. ..

```

```

        t1 = time.time()
        beta_results[sigma] = (lam_min, lam_mean)
        print(f"  σ={sigma:5.3f}: min λ_min={lam_min:.6f}, mean λ_min={lam_m
results[beta] = beta_results

print("\n\nSUMMARY TABLE (min λ_min):")
for beta in beta_list:
    print(f"\nβ = {beta}")
    for sigma in sigma_grid:
        lam_min, lam_mean = results[beta][sigma]
        print(f"  σ={sigma:5.3f}  min λ_min={lam_min: .6f}")

```

Starting β - σ Hessian convexity sweep...

```

=====
β = 0.5
=====

```

```

NameError                                Traceback (most recent call last)
/tmp/ipython-input-336311284.py in <cell line: 0>()
    23     for j, sigma in enumerate(sigma_grid):
    24         t0 = time.time()
--> 25         lam_min, lam_mean = estimate_lambda_min(
    26             L=L,
    27             beta=beta,

```

```

----- 1 frames -----
/tmp/ipython-input-1768347970.py in get_flat_action(L, beta, c0)
    16     n_params:         int
    17     """
--> 18     flat_action, unflatten, n_params = make_flat_funcs(L, beta, c0)
    19     flat_action_jit = jax.jit(flat_action)
    20     return flat_action_jit, unflatten, n_params

```

NameError: name 'make_flat_funcs' is not defined

